COURSE CODE/TITLE: CPE 010-CPE21S1 - Data Structures and Algorithms	**SECTION: CPE21S1**
DATE PERFORMED: 15/10/2026	**DATE SUBMITTED: 19/10/2026**
NAME: Mendoza, Roy Gabriel I. **Pilapil, Loreen Jae D.**	**INSTRUCTOR: Engr. Jimlord Quejado**

ACTIVITY NO. 13 [ACTIVITY NAME]

1. PROCEDURE OUTPUT

What procedures did you perform?

SCREENSHOT

Mendoza

```cpp
#include <iostream>
#include <vector>
#include <thread>
#include <chrono>

void print(int n, const std::string &str) {
    std::cout << "Printing integer: " << n << std::endl;
    std::cout << "Printing string: " << str << std::endl;
}

int main() {
    auto start :time_point<system_clock> = std::chrono::high_resolution_clock::now();

    std::thread t1( [>>] print, 10, "T.I.P.");
    t1.join();

    auto end :time_point<system_clock> = std::chrono::high_resolution_clock::now();

    auto duration = std::chrono::duration_cast<std::chrono::microseconds>( d: end - start);
    std::cout << "\nExecution time: " << duration.count() << " microseconds" << std::endl;

    return 0;
}
```

```
C:\Users\royga\CLionProjects\untitled1\HOA13p1\HOA13p1.exe
Printing integer: 10
Printing string: T.I.P.

Execution time: 6715 microseconds

Process finished with exit code 0
```

<table>
<tr><td></td><td>
<table>
<tr><td>Pilapil</td><td>

```cpp
#include <iostream>
#include <vector>
#include <thread>
#include <chrono>

void print(int n, const std::string &str) {
std::cout << "Printing integer: " << n << std::endl;
std::cout << "Printing string: " << str << std::endl;
}

int main() {
auto start = std::chrono::high_resolution_clock::now();

std::thread t1(print, 10, "T.I.P.");
t1.join();

auto end = std::chrono::high_resolution_clock::now();

auto duration = std::chrono::duration_cast<std::chrono::microseconds>(end - start);
std::cout << "\nExecution time: " << duration.count() << " microseconds" << std::endl;

return 0;
}
```

```
PROBLEMS  OUTPUT  DEBUG CONSOLE  TERMINAL  PORTS

r=mi'
Printing integer: 10
Printing string: T.I.P.
```

</td></tr>
</table>
</td></tr>
<tr><td>ANALYSIS</td><td>
<table>
<tr><td>Mendoza</td><td>This program applies the threading in our code. It demonstrates how the code is launched with a thread and parallelizes or makes it join back to the main program.</td></tr>
<tr><td>Pilapil</td><td>This program aims to show how long it took for the program to execute. It also joins a thread that prints two lines of texts to the console.</td></tr>
</table>
</td></tr>
</table>

Table 13-1. Simple One-Threaded Example

SCREENSHOT	

Mendoza

```cpp
#include <iostream>
#include <vector>
#include <thread>
#include <string>
#include <chrono>

void print(int n, const std::string &str) {
    std::string msg = std::to_string(n) + " : " + str;
    std::cout << msg << std::endl;
}

int main() {
    std::vector<std::string> s = {
        "T.I.P.",
        "Competent",
        "Computer",
        "Engineers"
    };

    std::vector<std::thread> threads;

    auto start :time_point<system_clock> = std::chrono::high_resolution_clock::now();

    for (size_t i = 0; i < s.size(); i++) {
        threads.push_back( x: std::thread( [>>] print, [>>] i, [>>] s[i]));
    }

    for (auto &th :thread & : threads) {
        th.join();
    }

    auto end :time_point<system_clock> = std::chrono::high_resolution_clock::now();
    auto duration = std::chrono::duration_cast<std::chrono::microseconds>( d: end - start);

    std::cout << "\nExecution time: " << duration.count() << " microseconds" << std::endl;

    return 0;
}
```

```
C:\Users\royga\CLionProjects\untitled1\HOA13p1\HOA13p2.exe
0 : T.I.P.1 : Competent

3 : Engineers
2 : Computer

Execution time: 7887 microseconds

Process finished with exit code 0
```

<table>
<tr>
<td></td>
<td>
<table>
<tr>
<td>Pilapil</td>
<td>
hoa13p2.cpp > main()

```cpp
#include <iostream>
#include <vector>
#include <thread>
#include <string>
#include <chrono>

void print(int n, const std::string &str) {
std::string msg = std::to_string(n) + " : " + str;
std::cout << msg << std::endl;
}

int main() {
std::vector<std::string> s = {
"T.I.P.",
"Competent",
"Computer",
"Engineers"
};

std::vector<std::thread> threads;

auto start = std::chrono::high_resolution_clock::now();

for (size_t i = 0; i < s.size(); i++) {
threads.push_back(std::thread(print, i, s[i]));
}

for (auto &th : threads) {
th.join();
}

auto end = std::chrono::high_resolution_clock::now();
auto duration = std::chrono::duration_cast<std::chrono::microseconds>(end - start);

std::cout << "\nExecution time: " << duration.count() << " microseconds" << std::endl;

return 0;
}
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

```
r=mi'
0 : T.I.P.1 : Competent

3 : Engineers
2 : Computer

Execution time: 30780 microseconds
PS C:\Users\TIPQC\Documents\CPE21S1>
```
</td>
</tr>
</table>
</td>
</tr>
<tr>
<td>ANALYSIS</td>
<td>
<table>
<tr>
<td>Mendoza</td>
<td>This program demonstrated how the code uses multithreading by parallelizing the code to make it run simultaneously for each thread and to make it work not step by step, but by task by task, and the output comes from what finished first from all the task that is being done.</td>
</tr>
<tr>
<td>Pilapil</td>
<td>This program aims to show how Multithreading works in C++ programming by ranking 4 items. We also added an extra feature that counts the time it took for the program to finish.</td>
</tr>
</table>
</td>
</tr>
</table>

Table 13-2. Multithreaded Example

2. ANSWERS TO SUPPLEMENTAL ACTIVITY

Copy and paste the questions from the lab and include your answers after.

Part A:

Demonstrate an understanding of parallelism, concurrency, and multithreading in C++ by answering the given questions. Use of supplementary materials to support answers must be cited as reference.

1. Write a definition of multithreading and its advantages/disadvantages

MENDOZA	PILAPIL
Multithreading is a a process of allowing a single process to execute multiple threads while sharing the same resources, but drastically improves the speed and responsiveness of the program, It can be used to handle like in web services, multiple requests, but at the cost of more resources taken and more complexity to execute a program if there are too many requests to be executed at the same time.	Multithreading is a programming technique where a single program runs multiple threads to perform different tasks. The advantages are improved performance, better multitasking, and efficiency. While the disadvantages are complex programming, uses additional system resources, and race conditions.

2. Rationalize the use of multithreading by providing at least 3 use-cases

MENDOZA	PILAPIL
1. In the application of a GUI, multithreading is important to keep the device responsive, by partitioning the work to others. 2. Another is web and network services, where possibly thousands of requests are done every moment, we need multithreading so that we would not run these sequentially. 3. Data processing is like machine learning, where we train models by breaking the task into independent sub tasks.	1. A browser uses multiple threads to load web pages, play videos, and process user input. 2. Games use multiple threads to handle graphics, sound, and physics calculations. 3. Downloading multiple files at the same time helps improve efficiency and allows users to continue using the application.

3. Differentiate between parallelism and concurrency.

MENDOZA	PILAPIL
Concurrency refers to the application's ability to manage multiple tasks at the same time. And parallelism is about running the tasks at the exact same instant which requires multiple cores rather than 1 like in concurrency.	Concurrency means handling multiple tasks within the same time, while parallelism means executing multiple tasks simultaneously.

PART B:

CODE

```cpp
#include <iostream>
#include <thread>

// GLobal variable
int globalVar = 0;

//Add function
void add(int value) {
    globalVar += value;
}

int main() {
    std::thread t1([>>]add, 10);
    std::thread t2([>>]add, 20);
    std::thread t3([>>]add, 30);

    // Display the value of global variable before joins
    std::cout << "Value before any join(): " << globalVar << std::endl;

    // T1.join()
    t1.join();
    // Display
    std::cout << "Value after T1.join(): " << globalVar << std::endl;

    // T2.join()
    t2.join();
    // Display
    std::cout << "Value after T2.join(): " << globalVar << std::endl;
```

```
    // T2.join()
    t2.join();
    // Display
    std::cout << "Value after T2.join(): " << globalVar << std::endl;

    // T3.join()
    t3.join();
    // Display (final confirmation)
    std::cout << "Value after T3.join(): " << globalVar << std::endl;

    return 0;
}
```

OUTPUT

```
"C:\Users\royga\CLionProjects\untitled1\HOA13p1\HOA13 supB.exe"
Value before any join(): 0
Value after T1.join(): 60
Value after T2.join(): 60
Value after T3.join(): 60

Process finished with exit code 0
```

ANALYSIS

MENDOZA	PILAPIL
The program initially shows the global variable as 0, after calling the T1.join(), the value instantly becomes 60, meaning that all three of the working threads had completed the task concurrently.	The program creates three threads that add 10, 20, and 30 to a shared global variable. It uses join() to wait for each thread to finish before continuing. The final value is 60, showing how multiple threads perform tasks within one program.

PART C:

```cpp
#include <iostream>
#include <cstdlib>
#include <ctime>
#include <thread>
#include <vector>

const int maxSize = 100;
int unsortedArr[maxSize];

void bubbleSort(int arr[], int n)
{
    for (int i = 0; i < n - 1; i++){
        for (int j = 0; j < n - i - 1; j++){
            if (arr[j] > arr[j + 1]){
                int temp = arr[j];
                arr[j] = arr[j + 1];
                arr[j + 1] = temp;
            }
        }
    }
}

void displayArr(int arr[])
{
    for (int i = 0; i < maxSize; i++){
        std::cout << arr[i] << " ";
    }
}

void countVotes(int arr[], int start, int end, int localCount[])
{
    for (int i = start; i < end; i++){
        localCount[arr[i]]++;
    }
}
```

```cpp
int main()
{
    srand(time(0));
    for (int i = 0; i < maxSize; i++){
        unsortedArr[i] = rand() % 5 + 1;
    }

    std::cout << "Unsorted Votes:\n";
    displayArr(unsortedArr);

    bubbleSort(unsortedArr, maxSize);

    std::cout << "\nSorted Votes:\n";
    displayArr(unsortedArr);

    //creates the threads
    const int numThreads = 4;
    int localCounts[numThreads][6] = {};
    std::vector<std::thread> threads;
    int chunk = maxSize / numThreads;

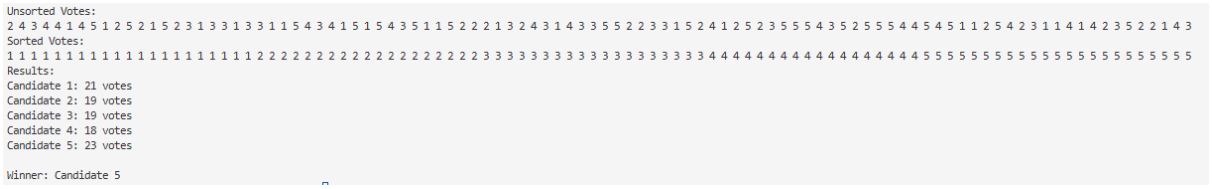
    for (int t = 0; t < numThreads; t++){
        int start = t * chunk;
        int end = (t == numThreads - 1) ? maxSize : start + chunk;
        threads.emplace_back(countVotes, unsortedArr, start, end, localCounts[t]);
    }

    for (auto& th : threads) th.join();
//combines the results from each thread
    int count[6] = {0};
    for (int t = 0; t < numThreads; t++){
        for (int c = 1; c <= 5; c++){
            count[c] += localCounts[t][c];
        }
    }

    std::cout << "\nResults:\n";
    std::cout << "Candidate 1: " << count[1] << " votes\n";
    std::cout << "Candidate 2: " << count[2] << " votes\n";
    std::cout << "Candidate 3: " << count[3] << " votes\n";
    std::cout << "Candidate 4: " << count[4] << " votes\n";
    std::cout << "Candidate 5: " << count[5] << " votes\n";

    int winner = 1;
    for (int i = 2; i <= 5; i++){
        if (count[i] > count[winner])
        {
            winner = i;
        }
    }
    std::cout << "\nWinner: Candidate " << winner << std::endl;

    return 0;
}
```



ANALYSIS

MENDOZA	PILAPIL
This implementation ensures thread safety by allocating an isolated row within the 2d array for each worker thread. This prevents the data taxes and memory conflicts that would occur if multiple threads tried to update a single shared counter at the same time.	The main purpose of multithreading here is to divide the counting task into smaller parts so multiple threads work on them simultaneously. In simple terms, instead of asking one person to count 100 votes, you ask 4 people to count 25 votes each. Once everyone finishes, you combine their results to determine the winner.

CONCLUSION

MENDOZA	PILAPIL
This lesson has shown me the idea and the processes regarding multi threading. Overall, the procedure built the main concept of the lesson, while the supplementary builds or helps us to understand it better by making us implement it. So far from my understanding, this lesson is one of the most important so far, because this lesson is like a backbone to all processes in programs to run simultaneously and efficiently in terms of time. As for how I did in this activity, I would say that I did well for my partner has also helped me understand the concepts that I am lacking or in need to be asked to the professor.	This activity successfully showed me how multithreading works. It's basically a programming technique where a program runs multiple threads to perform different tasks at the same time. The most challenging part of my supplementary activity part was finding previous activities where I could apply multithreading in a way that would improve the program's performance and efficiency. In the end, I chose the sorting program because it is where could apply multithreading and explore how it works alongside an existing algorithm.

3. ARTIFICIAL INTELLIGENCE (AI) USE DISCLOSURE STATEMENT

I hereby declare that artificial intelligence (AI) tools were used, if applicable, in the preparation of this academic work. The following indicates the nature and extent of AI assistance:

Tools Used: **None**

Examples: ChatGPT, Microsoft Copilot, Grammarly, QuillBot, etc.

Purpose of Use: **None**

Examples: grammar correction, idea generation, formatting, summarization, code assistance, explanation of concepts, etc.

Extent of Use: **None**

Explain how AI was used and confirm that it did not replace original thinking, analysis, authorship, or completion of the required work.

By submitting this activity, I affirm that my use of AI complies with T.I.P.'s AI usage policy and does not exceed the permitted limits for similarity, automated content generation, or academic assistance.

I understand that any false, incomplete, or misleading disclosure may be subject to investigation in accordance with due process and may result in sanctions for violation of existing T.I.P. policies.

Lab activities, templates, and related instructional materials shall not be reuploaded, reproduced, distributed, or shared with external organizations or third parties without the prior express written permission of the Technological Institute of the Philippines.

